

AeMS— 企业级综合网络管理系统

定位： 面向海量 Small Cell（小基站）网元设备的综合智能网管中枢前端，基于 Angular 20 构建，管理规模达数十万台 网元，覆盖设备监控、智能告警、日志分析、安全审计全链路。

一、技术栈全景

层级	技术选型	关键版本
框架	Angular (Standalone + Signals + 声明式控制流)	20.3
UI	Ng-Zorro + @axyom-ui 自研组件库 (table/form/acl/theme)	20.4
图表	ECharts (按需引入 Bar/Pie/Line/Tree)	5.x
地图	OpenLayers + GeoServer WMS	10.x
实时通信	STOMP over SockJS	7.x
状态管理	Angular Signals (signal/computed/effect)	—
路由	Hash 策略 + 自定义 LRU RouteReuseStrategy	—
样式	Less + BEM 命名规范	—
构建	Angular CLI + pnpm	10.x
工程化	ESLint + Prettier + Husky + lint-staged	—

二、系统架构设计

2.1 四层分层架构

Page Layer（页面壳） LayoutComponent ← HeaderComponent + SidebarComponent LoginComponent
Routes Layer（业务路由） – 12 个功能模块 dashboard manage alarm log setting report monitor event firmware cell ...
Share Layer（共享组件） – 14 个可复用业务组件 CardComponent ButtonGroupComponent CardFilterComponent PageTitleComponent TabCardComponent TimeRangeComponent SelectNeModelComponent InputPopComponent ...
Core Layer（核心基础设施） AuthService MenuService LoadingService StatusService StompService authInterceptor loadingInterceptor

userGuard RedirectGuard CustomReuseStrategy
Pagination<T> ROLE STORAGE utils
API Layer (声明式 HTTP 服务) – 装饰器驱动
AlarmService NeService CommonService MonitorService
DeviceLogService ParameterService ConfigService ...
继承 BaseApi, 使用 @GET/@POST/@PATH/@BODY/@QUERY 装饰器

2.2 模块规模统计

模块	页面数	组件数	API 服务数
manage (网元管理)	12	40+	8 (ne-list/ne-group/ne-model/profile/provision/bulk/reboot/rebalance)
alarm (告警管理)	6	15+	4 (alarm/rule/snmp/param)
log (日志管理)	4	4	5 (system/operation/security/event/param)
setting (系统设置)	4	30+	6 (config/user/permission/sftp/ldap/capacity)
monitor (节点监控)	2	4	2 (monitor/count)
dashboard (仪表盘)	1	3	2 (common/alarm)
report (报表)	1	1	1 (threshold)
event (事件)	1	1	—
firmware (固件)	1	1	—
合计	35+	100+	30+

2.3 数据流全链路





三、核心技术亮点（12 项）

3.1 声明式 API 服务层 — 装饰器驱动的 HTTP 抽象

解决的问题： 30+ 个 API 服务、200+ 个接口方法，如果每个都手动调用 **HttpClient**，会产生大量重复样板代码。

```
// =====  
// 传统方式 - 每个接口都需要手动拼装 HttpClient 请求  
// =====  
searchAlarm(type, query, data): Observable<Page<AlarmResult>> {  
  return this.http.post<Page<AlarmResult>>(  
    `/hems-web-ui/alarm/${type}/search`, data, { params: { ...query } }  
  );  
}  
  
// =====  
// 本项目方式 - 装饰器声明式定义，零样板代码  
// =====  
@POST('/:alarmType/search')  
searchAlarm(  
  @PATH('alarmType') type: AlarmType,  
  @PAYLOAD query: AlarmQueryDto,  
  @BODY data: Filter,  
) : Observable<Page<AlarmResult>> {  
  return null as never; // 运行时由 BaseApi 装饰器代理  
}
```

装饰器体系：

装饰器	作用	示例
@BaseUrl	设置服务基础路径	@BaseUrl('/hems-web-ui/alarm')
@GET/@POST/@DELETE	定义 HTTP 方法和 URL 模板	@POST('/:type/search')
@PATH	URL 路径参数绑定	@PATH('type') type: AlarmType
@QUERY	Query String 参数绑定	@QUERY('id') id: number
@BODY	Request Body 绑定	@BODY data: Filter
@PAYLOAD	序列化为 Query String	@PAYLOAD query: AlarmQueryDto

深度分析：

- **编译期类型安全：**泛型返回值 `Observable<Page<T>>` + 参数装饰器，TypeScript 编译器可捕获接口签名不匹配
- **URL 模板引擎：**`:param` 占位符在运行时自动替换为 `@PATH` 参数值
- **零运行时开销：**方法体返回 `null as never`，实际 HTTP 调用由 `BaseApi` 的 Proxy/Reflect 拦截器透明代理
- **API 与后端 1:1 映射：**每个装饰器方法对应一个 REST 端点，便于维护和文档生成

面试展开点： "装饰器元数据在运行时如何被读取？BaseApi 如何实现方法拦截？如何处理嵌套 PATH 参数？ "

3.2 LRU 路由缓存策略 — 页面级视图复用

解决的问题： 用户在 Active List → 设备详情 → 告警历史 → 返回列表 时，列表页需要重新加载数据并丢失滚动位置。

```
export class CustomReuseStrategy implements RouteReuseStrategy {
  private static MAX_CACHE_SIZE = 6;
  static handlers: { [key: string]: CacheItem } = {};

  // ① 判断是否需要缓存（由路由 data.keepAlive 控制）
  shouldDetach(route: ActivatedRouteSnapshot): boolean {
    return !!route.data?.['keepAlive'];
  }

  // ② 存储缓存（含组件引用 + 滚动位置 + 模块名）
  store(route: ActivatedRouteSnapshot, handle: DetachedRouteHandle | null) {
    const scrollContainer = document.querySelector('.ant-table-body');
    CustomReuseStrategy.handlers[key] = {
      handle,
      moduleName: route.data?.['moduleName'],
      scrollTop: scrollContainer?.scrollTop ?? 0,
    };
  }

  // ③ 恢复缓存（LRU 刷新：删除后重新插入，移到末尾）
  retrieve(route: ActivatedRouteSnapshot) {
    const item = CustomReuseStrategy.handlers[key];
```

```

if (item) {
  delete CustomReuseStrategy.handlers[key]; // 先删
  CustomReuseStrategy.handlers[key] = item; // 后插入 → 移到末尾
  setTimeout(() => { // 异步恢复滚动
    document.querySelector('.ant-table-body')!.scrollTop = item.scrollTop;
  }, 0);
  return item.handle;
}
return null;
}

// ④ 缓存上限保护：超出 6 个时销毁最早的
private ensureCacheLimit() {
  if (Object.keys(handlers).length >= MAX_CACHE_SIZE) {
    const oldkey = Object.keys(handlers)[0];
    destroyHandle(handlers[oldkey].handle);
    delete handlers[oldkey];
  }
}

// ⑤ 跨模块清理：切换大模块时清理其他模块缓存
static deleteOtherModuleCache(targetModuleName: string) {
  Object.keys(handlers).forEach(key => {
    if (handlers[key].moduleName !== targetModuleName) {
      destroyHandle(handlers[key].handle);
      delete handlers[key];
    }
  });
}
}

```

设计亮点：

- **LRU 淘汰**：最多 6 个页面，超出自动销毁最久未访问的
- **滚动位置恢复**：表格 `.ant-table-body` 的 `scrollTop` 跨导航保持
- **模块级隔离**：切换到 Setting 模块时自动清理 Manage 模块缓存
- **手动销毁**：`componentRef.destroy()` 确保 Detached 组件的 `ngOnDestroy` 被调用
- **声明式控制**：路由 `data: { keepAlive: true, moduleName: 'manage' }` 声明缓存策略

3.3 位编码权限体系 — 100+ 权限点的 RBAC

解决的问题：系统有 100+ 个操作权限点，需要同时控制菜单可见性、路由访问、按钮启用状态。

```

// =====
// 权限常量树 - 树形结构定义所有权限
// =====
export const ROLE = {
  cell: {
    M: '3', // 模块编号
    active: {

```

```

    M: '1',
    EXPORT: { M: '1' },          // 权限码 = "311"
    maintenance: {
      general: {
        GRACEFUL: { M: '1' },    // 权限码 = "31311"
        FORCEFUL: { M: '2' },    // 权限码 = "31312"
      }
    }
  },
  alarm: { M: '4', history: { M: '1', DELETE: { M: '1' } } }, // "411"
  log: { M: '7', operation: { M: '2', EXPORT: { M: '1' } } }, // "721"
};

// =====
// 自动生成权限码 → 路径描述映射（递归遍历）
// =====
function getLeafNodesWithPath(): Map<string, string> {
  // "311" → "Small Cell Management > Small Cell List > EXPORT"
  // "412" → "Alarm Management > Active Alarms > ACK"
  // 369 个旧权限码 + 新权限码全部映射
}

```

三层权限控制机制：

层级	实现	控制粒度
菜单层	<code>MenuService.handleAcl()</code> 递归过滤	整个菜单项隐藏/显示
路由层	<code>RedirectGuard.canActivateChild()</code>	URL 级访问拦截
按钮层	<code>ButtonGroupComponent</code> + <code>ACLService.can()</code>	单个按钮禁用/启用

```

// 菜单动态裁剪 - 无权限子树整体隐藏
handleAcl(menus: SidebarMenu[]): SidebarMenu[] {
  return menus.filter(x => {
    if (x.children?.length) {
      x.children = this.handleAcl(x.children);
      return x.children.length > 0; // 子节点全部无权限 → 父节点也隐藏
    }
    return this.acl.can(this.getAclRole(x));
  });
}

// 按钮级 ACL - 每个按钮独立控制
<ButtonGroup [action]="[
  { label: 'Add', acl: ['Alarm Management_Configuration_Alarm Rules_W'], fun: add() },
  { label: 'Delete', acl: ['Alarm Management_Configuration_Alarm Rules_W'], fun: delete() },
]" />

```

向后兼容设计：

- `old_role.ts` 包含 369 个旧权限码映射
- `ROLE_LIST` 自动合并新旧映射，后端通过 `public/operation_log.json` 同步

3.4 十万级设备地图 — OpenLayers + BBOX 裁剪 + 聚合

解决的问题： Dashboard 地图需要展示数十万台设备的实时状态，直接渲染会导致浏览器崩溃。

```
// =====  
// 性能核心：视口裁剪 - 只渲染当前可见区域的设备点  
// =====  
private filterBBOXData(data: HeNB[]): HeNB[] {  
    const extent = view.calculateExtent(map.getSize()); // 当前视口  
    const bl = toLonLat(getBottomLeft(extent), PROJ_CODE);  
    const tr = toLonLat(getTopRight(extent), PROJ_CODE);  
    return data.filter(item => {  
        const lng = Number(item.longitude) / COORD_SCALE;  
        const lat = Number(item.latitude) / COORD_SCALE;  
        return lng >= bl[0] && lng <= tr[0] && lat >= bl[1] && lat <= tr[1];  
    });  
}  
  
// =====  
// 聚合显示：同区域多设备合并为一个点，显示数量  
// =====  
const count = data?.neNumOfSameMarketAndOnlineStatus || 0;  
const isCluster = count > 0;  
const radius = isCluster ? 12 : 6; // 聚合点更大  
if (isCluster) {  
    style.setText(new Text({ text: count.toString(), fill: '#fff' }));  
}  
  
// =====  
// 自定义投影 - 注册 EPSG:900913 并建立双向转换  
// =====  
private registerCustomProjection() {  
    const proj = new Projection({ code: 'EPSG:900913', units: 'm' });  
    addProjection(proj);  
    addCoordinateTransforms(EPSPG4326, proj,  
        getTransform(EPSPG4326, EPSG3857),  
        getTransform(EPSPG3857, EPSG4326)  
    );  
}
```

性能优化四板斧：

策略	实现	效果
BBOX 视口裁剪	<code>filterBBOXData()</code> 只渲染视口内 Feature	10 万设备 → 仅渲染百级点位
聚合显示	同 Market 同状态设备合并为一个点	视口内点位数进一步降低

策略	实现	效果
数据缓存	<code>dataCache: Map<string, HeNB></code> 全量缓存	缩放平移不重新请求后端
<code>moveend</code> 懒刷新	<code>refreshVisibleFeatures()</code> 仅在移动结束时触发	避免拖拽过程中的频繁重绘

地图状态持久化:

```
// sessionStorage 保存/恢复地图视图
sessionStorage.setItem('osm-map-view-state', JSON.stringify({
  market: currentMarket, zoom: view.getZoom(), center: toLonLat(view.getCenter())
}));

// Market 切换防抖 - 切换时暂停持久化, 避免保存旧状态
this.mapStatePersistenceEnabled = false;
sessionStorage.removeItem(MAP_STATE_STORAGE_KEY);
```

3.5 精确 Loading 管理 — 请求级粒度追踪

解决的问题: 全局 Loading 条无法区分多个并发请求, 用户看到 Loading 闪烁。

```
// =====
// LoadingService - Signal 驱动的请求计数器
// =====
@Injectable({ providedIn: 'root' })
export class LoadingService {
  private readonly cache = signal<string[]>([]);

  start(key: string) { this.cache.set([...this.cache(), key]); }
  stop(key: string) { this.cache.set(this.cache().filter(x => x !== key)); }

  getLoading(key: string): boolean {
    const regex = this.buildRegexp(key); // 智能构建正则
    return this.cache().find(x => regex.test(x)) !== null;
  }

  // 支持 "POST /path" 和 "/path" 两种格式
  private buildRegexp(url: string): RegExp {
    // "POST /config/nes/advanceSearch"
    // → /^POST /hems-web-ui(/w+)*?\/config\/nes\/advanceSearch$/
  }
}

// =====
// 拦截器: 每个 HTTP 请求自动追踪
// =====
export const loadingInterceptor: HttpInterceptorFn = (req, next) => {
  if (req.url.startsWith('assets')) return next(req); // 跳过静态资源
  const key = `${req.method} ${req.url.split('?')[0]}`;
  service.start(key);
  return next(req).pipe(finalize(() => service.stop(key)));
}
```



```

};

// =====
// 按钮组件自动关联 Loading 状态
// =====
readonly action = signal<CardAction[]>([
  {
    label: 'Refresh',
    key: 'POST /config/nes/advanceSearch', // ← 与拦截器的 key 精确匹配
    fun: () => this.refresh(),
  },
]);
// ButtonGroupComponent 内部:
// processed.isLoading = () => this.loading.getLoading(processed.key!);

```

核心设计:

- **请求级粒度**: 每个 HTTP 请求独立追踪, 精确到 `METHOD /path`
- **正则智能匹配**: `buildRegexp()` 自动处理 API 前缀, 支持模糊匹配
- **Signal 响应式**: `cache` 是 signal, 状态变化自动触发 `OnPush` 更新
- **与按钮组件联动**: `CardAction.key` 直接关联请求 URL, 零配置自动显示 Loading

3.6 SessionStorage 装饰器 — 零侵入状态持久化

```

// 装饰器定义 - 利用 Object.defineProperty 重写 getter/setter
export function SessionStorage(key?: string, parse = true) {
  return function (target: unknown, propertyKey: string) {
    const storageKey = key || propertyKey;
    Object.defineProperty(target, propertyKey, {
      get: () => {
        const value = sessionStorage.getItem(storageKey);
        return value ? (parse ? JSON.parse(value) : value) : null;
      },
      set: (newVal: unknown) => {
        sessionStorage.setItem(storageKey, JSON.stringify(newVal));
      },
    });
  };
}

// 使用 - 一行代码实现静态属性的 sessionStorage 持久化
export class STORAGE {
  @SessionStorage() // key='username', parse=true
  static username = '';

  @SessionStorage('userId', false) // key='userId', parse=false
  static userId = '-1';

  @SessionStorage('ine') // key='ine', parse=true
  static is_near_expired = false;
}

```

```

}

// 业务代码中直接读写，完全不感知 sessionStorage
STORAGE.username = 'admin'; // 自动 sessionStorage.setItem('username', 'admin')
console.log(STORAGE.username); // 自动 sessionStorage.getItem → JSON.parse

```

技术深度：

- **TypeScript 装饰器**：Stage 3 提案前的 experimental 装饰器，作用于静态属性
- **透明代理**：Object.defineProperty 重写 getter/setter，调用方无感知
- **JSON 自动序列化**：parse 参数控制是否自动 JSON.parse/stringify
- **页面刷新保持**：用户登录后 STORAGE.username = res.user.username，F5 刷新后仍可读取

3.7 STOMP WebSocket 实时通信

```

// StompService - 封装 STOMP over SockJS
export class StompService {
  stompClient = new RxStomp();
  constructor(destination: string, topic = '') {
    const socket = new SockJS(`/hems-web-ui/${destination}`);
    this.stompClient.configure({
      reconnectDelay: 5000, // 断线后 5 秒自动重连
      heartbeatIncoming: 4000, // 4 秒心跳检测
      heartbeatOutgoing: 4000,
      websocketFactory: () => socket,
    });
    this.stompClient.activate();
  }
  watch() { return this.stompClient.watch('/topic/' + this.topic); }
}

// =====
// 场景 1: 强制登出推送
// =====
// LayoutComponent.ngOnInit()
this.stomp = new StompService('forcedLogout');
this.stomp.watch().subscribe(data => {
  if (data.body == STORAGE.userId) {
    this.auth.logout(); // 管理员踢出当前用户
  }
});

// =====
// 场景 2: 参数树实时刷新状态
// =====
// ParameterTreeComponent.ngOnInit()
this.stomp = new StompService('parameterMonitor', 'parameterMonitor/' + identity);
this.stomp.watch().subscribe(msg => {
  if (msg.body === 'refreshing') {

```

```

    this.flag = { text: 'Refreshing', color: '#eb7836' };
  } else {
    this.flag = { text: `Refresh Time: ${new Date(parseInt(msg.body)).toISOString()}`,
color: '#49b77a' };
  }
});

```

三个 WebSocket 使用场景：

场景	Destination	用途
强制登出	<code>forcedLogout</code>	管理员踢出在线用户
参数监控	<code>parameterMonitor/{identity}</code>	参数树刷新状态实时推送
通用推送	按需创建	扩展到告警推送等场景

3.8 异步导出 — RxJS 流式轮询 + 流式下载

解决的问题： 导出 10 万条告警数据，后端异步生成文件，前端需要轮询状态并下载。

```

export() {
  this.neService.exportNeList(data).pipe(
    mergeMap(exportRes =>
      this.common.downloadNeList(exportRes.requestId).pipe(
        expand(res => {
          if (res.status === 'executing') {
            return timer(2000).pipe( // 2 秒间隔轮询
              mergeMap(() => this.common.downloadNeList(exportRes.requestId))
            );
          }
          return throwError(() => new Error(res.status));
        }),
        takeWhile(res => res.status === 'executing', true), // 执行中继续，完成/失败停止
      )
    ),
  ).subscribe(res => {
    if (res.status === 'success') {
      downloadFileByRequestId(res.requestId); // 浏览器直接下载，不经内存
    }
  });
}

```

// downloadFile - 通过 <a> 标签触发浏览器下载，不加载到内存

```

export function downloadFile(url: string, filename = '') {
  const a = document.createElement('a');
  a.href = url;
  if (filename) a.download = filename;
  document.body.appendChild(a);
  a.click();
  document.body.removeChild(a);
}

```

```
}
```

RxJS 操作符链分析:

- `mergeMap`: 将导出请求的结果展平为下载状态轮询流
- `expand`: 递归展开, 每次返回新的轮询 Observable
- `timer(2000)`: 2 秒间隔, 避免过于频繁的请求
- `takeWhile(..., true)`: `executing` 继续轮询, `success/failed` 停止
- `downloadFile`: 零内存占用的浏览器原生下载

3.9 GeoHA 双活架构 — 无状态网元归属判断

解决的问题: 两台 AeMS 服务器双活部署, 网元分散在两台服务器上, 操作前需判断归属。

```
@Injectable({ providedIn: 'root' })
export class StatusService {
  // 启动时预加载系统配置
  constructor() {
    this.common.roleAndLocation().subscribe(data => {
      this.location.set(data.location);
      this.role.set(data.role);
      this.dbIdOffset = parseInt(data.dbIdOffset);
      this.dbIdIncrement = parseInt(data.dbIdIncrement);
      this.geoHaEnable = data.geoHaEnable === 'true';
    });
  }

  // 基于 ID 哈希的无状态归属判断
  isRemoteNe(id: number) {
    return this.geoHaEnable &&
      this.dbIdOffset % this.dbIdIncrement !== id % this.dbIdIncrement;
  }

  isLocalNe(id: number) {
    if (this.isRemoteNe(id)) {
      this.modal.warning({
        nzContent: 'This NE is in remote AeMS. Please go to remote AeMS for operation.'
      });
      return false;
    }
    return true;
  }
}

// 使用 - 所有网元操作前必须检查
reboot(effectiveStrategy: number) {
  if (this.neData.status == 0) {
    this.message.warning("can't reboot offline Small Cell.");
    return;
  }
}
```

```
// ...
}

// BaseMenuService.gotoByGeo() - 封装地理检查
gotoByGeo(row: NeTree, url: string) {
  if (this.status.isLocalNe(row.id)) {
    this.router.navigateByUrl(url);
  }
}
```

分布式设计要点:

- **无状态判断**: `id % dbIdIncrement` 公式无需查询数据库, $O(1)$ 复杂度
- **前置拦截**: `isLocalNe()` 在所有操作前调用, 远程网元弹出警告
- **降级策略**: `geoHaEnable = false` 时所有网元都认为是本地网元

3.10 分页基类抽象 — Pagination<T> 泛型复用

解决的问题: 10+ 个列表页面都有分页、过滤、搜索、刷新逻辑, 需要统一抽象。

```
@Directive()
export class Pagination<T = unknown> implements OnInit {
  readonly page = signal<AxyomPage>(new AxyomPage({ pageSize: 100 }));
  readonly cols = signal<AxyomColumns<T>>([]);
  readonly origin = signal<T[]>([]);
  readonly selected = signal<T[]>([]);
  readonly searchKeyword = signal('');

  // computed 派生: 搜索关键词变化自动过滤, 不触发 HTTP 请求
  readonly rows = computed(() => {
    const keyword = this.searchKeyword();
    if (!keyword) return this.origin();
    return tableFilter(keyword, this.origin(), this.cols(), this.tableConfig);
  });

  // computed 派生: 分页查询参数自动生成
  readonly getQueryStr = computed(() => {
    const p = this.page();
    return `{"pageNum":${p.pageIndex + 1},"pageSize":${p.pageSize}}`;
  });

  refresh(): void { /* 子类实现 */ }
  filter(value: string): void { this.searchKeyword.set(value?.trim() || ''); }
}

// 子类只需实现 refresh()
export class ActiveListComponent extends Pagination<NeTree> {
  override refresh() {
    this.neService.getNeList(this.data.filter, this.getQueryStr()).subscribe(res => {
      this.origin.set(res.result);
      this.page.update(p => new AxyomPage({ ...p, total: res.total }));
    });
  }
}
```

```
    });
  }
}
```

继承体系:

子类	数据类型	功能
ActiveListComponent	NeTree	网元列表 (24+ 列、树形展开)
RebalanceTaskComponent	RebalanceTask	再平衡任务列表
NodeComponent	Status	节点监控列表 (自动 60 秒刷新)
RuleComponent	AlarmRule	告警规则列表
10+ 个其他列表	各类型	统一分页/过滤/搜索体验

3.11 声明式表单配置 — @axyom-ui/form 体系

解决的问题: 30+ 个表单页面 (搜索、编辑、配置), 表单定义需要统一规范。

```
// =====
// Active List 搜索表单 - 33 个字段的声明式定义
// =====

export function getForm(): FormBase[] {
  return [
    new StringUnit({ key: 'comments', label: 'Comments / Memo' }),
    new SelectUnit({ key: 'oui', label: 'OUI', options: [] }),
    new StringUnit({ key: 'macId', label: 'MAC ID' }),
    new SelectUnit({ key: 'status', label: 'Femto Status', options: [
      { label: 'online', value: '1' },
      { label: 'offline', value: '0' },
    ]}),
    // ... 33 个字段
  ];
}

// 动态更新下拉选项
export function updateOption(fbs: FormBase[], option: NeOption) {
  fbs.forEach(x => {
    if (x instanceof SelectUnit) {
      switch (x.key) {
        case 'oui': x.options = toOptions(option.oui); break;
        case 'marketName': x.options = toOptions(option.marketName); break;
        // ...
      }
    }
  });
}

// =====
```

```
// 弹窗表单 - DialogModal.builder() 链式调用
// =====
this.modal.confirm(
  DialogModal.builder()
    .setTitle('Reboot Small Cell')
    .setContent('Do you want to reboot this Small Cell?')
    .setOkDanger(false)
    .setOk(() => this.neOperationService.reboot({ id, effectiveStrategy, M })).pipe(
      tap({ next: () => this.message.success('success!') })
    )
);

// FormModal - 带表单的弹窗
this.modal.openModal(
  new FormModal({
    title: 'PCI Selection',
    fbs: [new SwitchUnit({ key: 'mode', label: 'SELECTION', value: x.mode == '1' })],
    resetText: '',
    onOk: (val) => this.neOperationService.setPciSelection(identity, val.mode ? 1 : 0),
  })
);
```

表单元类型:

类型	用途	特性
StringUnit	文本输入	支持 placeholder、disabled
SelectUnit	下拉选择	options 动态更新
SwitchUnit	开关	布尔值绑定
NumberUnit	数字输入	precision 精度控制
PasswordUnit	密码输入	自动掩码
DatePickerUnit	日期选择	disabledDate/disabledTime

3.12 全局搜索 — 防抖 + 自动补全

```
export class SearchComponent implements OnInit {
  readonly searchType = signal('mac');
  readonly searchValue = signal('');
  readonly listOfOption = signal<string[]>([]);
  private searchChange$ = new BehaviorSubject('');

  ngOnInit() {
    this.searchChange$.pipe(debounceTime(200)).subscribe(identity => {
      if (identity) {
        this.alarmService.getIdentityList(identity, getPageStr(0, 100))
          .subscribe(page => this.listOfOption.set(page.result));
      }
    });
  }
}
```

```
    }
  });
}

// 搜索 → 跳转到 Active List 并携带过滤条件
onSearch() {
  const data: NeCache = { filter: { [this.searchType()]: this.searchValue() }, size: 100,
index: 0 };
  sessionStorage.setItem('neData', JSON.stringify(data));
  this.router.navigate(['/manage/active'], { queryParams: { back: Date.now() } });
}
}
```

设计要点：

- **200ms 防抖**：避免输入过快时频繁请求
- **搜索类型切换**：支持 MAC / 序列号 / CorrelationHandle / Identity 四种搜索维度
- **SessionStorage 中转**：搜索条件写入 sessionStorage，Active List 读取后执行过滤
- **路由跳转 + QueryParams**：back: timestamp 触发 Active List 的 queryParamMap 订阅，执行数据刷新

四、项目难点与解决方案

4.1 海量网元表格的 24+ 列性能

难点： Active List 展示 24+ 列、支持分页/过滤/右键菜单/树形展开（LTE/NR 双模）。

解决方案矩阵：

层面	措施	效果
变更检测	Signal + OnPush	最小化重渲染范围
数据过滤	computed() 派生 rows	前端搜索零 HTTP 请求
路由复用	CustomReuseStrategy	切换返回不重新请求
状态恢复	SessionStorage	分页/过滤/滚动位置保持
基类抽象	Pagination<T>	代码复用，子类只写 refresh()
表单配置	FormBase 声明式	33 个字段统一管理

4.2 多模块路由缓存的内存管理

难点： 12 个功能模块、50+ 个子路由，全部缓存会导致内存溢出。

解决方案：

- LRU 淘汰（上限 6）+ 模块级清理 + 手动 destroy + 滚动位置分离存储

4.3 复杂权限的三级联动

难点：100+ 权限点需同时控制菜单/路由/按钮。

解决方案：

- 位编码常量树 → 自动生成映射表 → 菜单递归过滤 → 路由 Guard → 按钮 ACL

4.4 GeoHA 双活的网元操作安全

难点：两台 AeMS 双活，误操作远程网元会导致数据不一致。

解决方案：

- ID 哈希分片 O(1) 判断 + isLocalNe() 前置拦截 + 弹窗警告

五、架构优化总结

5.1 框架升级收益

维度	Angular 17→20 升级收益
状态管理	BehaviorSubject → Signal，API 更简洁、自动依赖跟踪
组件定义	NgModule → Standalone，Tree-shaking 更优
模板语法	*ngIf → @if，编译期优化、可读性提升
输入绑定	@Input() → input()，Signal 原生支持
生命周期	ngOnDestroy → takeUntilDestroyed()，内存安全

5.2 性能优化清单

优化项	措施	量化收益
ECharts 体积	按需引入 Bar/Pie/Line/Tree	减少约 60%
地图渲染	BBOX 裁剪 + 聚合	10 万 → 百级点位
Loading 精度	METHOD /path 级追踪	消除全局闪烁
Zone.js	eventCoalescing: true	减少变更检测次数
内联样式	inlineStyle: true	减少 HTTP 请求
API 层	装饰器声明式	消除 200+ 个方法的样板代码

六、面试技术问答（15 题）

Q1: 请介绍项目的整体架构设计

答：四层架构 — API 层 (装饰器声明式 HTTP) → Core 层 (全局服务/拦截器/守卫) → Routes 层 (业务组件) → Share 层 (14 个可复用组件)。关键决策：Angular 20 Signals 状态管理、Standalone 组件按需加载、装饰器驱动的 API 服务、LRU 路由缓存。

Q2: 如何处理十万台设备的地图性能？

答：四级优化：① BBOX 视口裁剪只渲染可见区域；② 聚合点合并同 Market 设备；③ dataCache 全量缓存避免重复请求；④ moveend 懒刷新避免拖拽重绘。自定义 EPSG:900913 投影 + GeoServer WMS 底图分层渲染。

Q3: LRU 路由缓存如何实现？

答：实现 RouteReuseStrategy 四个方法：shouldDetach 判断是否缓存、store 存储组件引用+滚动位置、retrieve 恢复并 LRU 刷新（先删后插）、ensureCacheLimit 超 6 个销毁最老。deleteOtherModuleCache 跨模块清理。

Q4: 权限体系的位编码设计原理？

答：树形常量树，每层节点 M 值拼接为权限码（如 311=cell>active>EXPORT）。getLeafNodesWithPath() 递归生成 Map<码, 路径>。三层控制：MenuService 菜单过滤、RedirectGuard 路由拦截、ButtonGroupComponent 按钮 ACL。

Q5: 声明式 API 服务的实现原理？

答：@axyom-ui/theme 的 BaseApi + 装饰器。@GET/@POST 定义 HTTP 方法和 URL 模板，@PATH/@QUERY/@BODY/@PAYLOAD 标记参数绑定。方法体返回 null as never，运行时 Proxy/Reflect 拦截器代理实际 HttpClient 调用。

Q6: Loading 状态如何精确追踪？

答：LoadingService 用 signal<string[]> 存储活跃请求 key。loadingInterceptor 每个请求 start/stop。getLoading(key) 用正则匹配。ButtonGroupComponent 的 CardAction.key 与请求 URL 关联，isLoading 回调自动判断。

Q7: 异步导出的 RxJS 操作符链？

答：exportNeList 返回 requestId → mergeMap 展平为轮询流 → expand 递归展开 → timer(2000) 2 秒间隔 → takeWhile 控制终止。downloadFile 通过 [标签触发浏览器下载，零内存占用](#)。

Q8: GeoHA 双活如何判断网元归属？

答： $id \% dbIdIncrement !== dbIdOffset \% dbIdIncrement$ 则为远程网元。O(1) 无状态判断，无需查询数据库。isLocalNe() 前置拦截所有操作，远程网元弹出警告。

Q9: sessionStorage 装饰器的实现原理？

答：TypeScript 装饰器 + Object.defineProperty 重写 getter/setter。getter 从 sessionStorage 读取并 JSON.parse，setter 写入时 JSON.stringify。业务代码直接读写静态属性，完全不感知存储层。

Q10: 项目中 Angular 20 的新特性应用？

答：① Signals (signal/computed/effect) 状态管理；② input()/input.required() Signal Inputs；③ viewChild.required()；④ @if/@for 声明式控制流；⑤ Standalone 组件无 NgModule；⑥ 函数式守卫 CanActivateFn；⑦ 函数式拦截器 HttpInterceptorFn；⑧ takeUntilDestroyed() 内置销毁管理。

七、右键菜单体系 — 装饰器模式的业务编排

7.1 菜单架构：组合模式

BaseMenuService (基础路由 + 面包屑 + 刷新事件)

```
|
|— ActiveListMenu (主菜单聚合器)
|   |— MaintenanceMenu (设备维护子菜单)
|   |— OperationMenu (设备操作子菜单)
|   |— RadiMenu (射频管理子菜单)
|   |— LogMenu (日志管理子菜单)
|
|— refresh$ Subject<{ event, action }> (跨菜单通信)
```

// ActiveListMenu - 聚合所有子菜单

```
getMenus(): AxyomMenu<NeTree>[] {
  return [
    { label: 'Alarm Management', children: [...] },
    ...this.maintenance.getMenus(),    // 维护子菜单
    ...this.operation.getMenus(),       // 操作子菜单
    ...this.log.getMenus(),             // 日志子菜单
    { label: 'Parameter Management', fun: (row) => this.service.goto(...) },
    ...this.radio.getMenus(),           // 射频子菜单
    { label: 'Trace', children: [...] },
    { label: 'Location Management', children: [...] } ],
};
```

// BaseMenuService - 跨菜单通信枢纽

```
refresh$ = new Subject<{ event: string; action: boolean }>();
```

// ActiveListComponent 订阅刷新事件

```
this.baseMenuService.refresh$
  .pipe(takeUntilDestroyed(this.destroyRef))
  .subscribe(ev => { if (ev.action) this.refresh(); });
```

设计模式分析：

- 组合模式 (Composite): `AxyomMenu<T>` 树形结构，支持任意层级嵌套

- **观察者模式**: `refresh$` Subject 实现菜单操作后自动刷新列表
- **策略模式**: 每个菜单服务独立封装业务逻辑, 主菜单只负责聚合
- **模板方法**: `CellBase.init()` / `LogBase.init()` 定义子类扩展点

7.2 操作前的安全检查链

```
// 每个操作都经过多层安全检查
private syncAlarm(row: NeTree) {
  // ① GeoHA 检查 - 远程网元拦截
  if (!isOnlineLocal) {
    this.modal.warning({ nzContent: 'NE is in remote AeMS...' });
    return;
  }
  // ② 执行操作
  this.alarmService.syncConfigs(row.identity, isOnlineLocal).subscribe(...);
}

private reboot(effectiveStrategy: number) {
  // ① 设备状态检查 - 离线设备禁止重启
  if (this.neData.status == 0) {
    this.message.warning('can't reboot offline Small Cell. ');
    return;
  }
  // ② GeoHA 检查
  // ③ 确认弹窗
  // ④ 执行操作
}

private lock(ne: NeTree) {
  // ① GeoHA 检查
  // ② Loading 提示
  const msg = this.message.loading(`Disabling NeID:${ne.id} Radio...`, { nzDuration: 0 });
  // ③ 执行操作 + finalize 移除 Loading
  this.neOperationService.lock({ neId: String(ne.id) })
    .pipe(finalize(() => this.message.remove(msg.messageId)))
    .subscribe(() => this.menu.refresh$.next({ event: 'lock', action: true }));
}
```

安全检查链: GeoHA 归属 → 设备状态 → 确认弹窗 → Loading 提示 → 执行操作 → 刷新列表

八、继承体系设计 — 模板方法模式

8.1 CellBase — 设备详情页基类

```
@Directive()
export class CellBase implements OnInit {
  neId = -1;
  identity = '';
  neData: Partial<NeInfo> = {};
```

```

breadcrumbs = this.baseMenu.breadcrumbs;
backUrl = this.baseMenu.backUrl;

ngOnInit() {
  this.route.paramMap.subscribe(paramMap => {
    this.neId = parseInt(paramMap.get('id')!);
    this.identity = paramMap.get('identity')!;
    this.breadcrumbs.push({ text: this.identity });
    this.refreshNeInfo(); // 模板方法
  });
}

refreshNeInfo() {
  this.common.getNeInfoByNeId(this.neId).subscribe(neInfo => {
    this.neData = neInfo;
    this.init(); // 扩展点 - 子类重写
  });
}

init() { /* 子类扩展 */ }
}

// 8 个子类继承 CellBase
GeneralComponent extends CellBase // 通用管理（重启/恢复出厂/频段选择）
UpgradeComponent extends CellBase // 固件升级
DiagnosticsComponent extends CellBase // 诊断任务

```

8.2 LogBase — 日志页基类

```

@Directive()
export class LogBase implements OnInit {
  id = 0;
  neIdentity!: string;
  breadcrumbs = this.baseMenu.breadcrumbs;

  ngOnInit() {
    this.route.paramMap.subscribe(paramMap => {
      this.id = parseInt(paramMap.get('id')!);
      this.neIdentity = paramMap.get('identity')!;
      this.breadcrumbs.push({ text: this.neIdentity });
      this.init(); // 扩展点
    });
  }

  init() { /* 子类扩展 */ }
}

// 2 个子类继承 LogBase
DeviceLogComponent extends LogBase // 设备日志
CwmpLogComponent extends LogBase // CWMP 日志

```

8.3 FileServerBase — 文件服务器配置基类

```
@Directive()
export class FileServerBase implements OnInit {
  readonly fbs = signal<FormBase[]>([]);
  readonly form = signal<FormGroup>(new FormGroup({}));

  constructor(private type: FileServer['type']) {} // 'autolog' | 'packetcapture' | 'deviceLog'

  ngOnInit() {
    this.createFormUnits(); // 模板方法
    this.refresh();
  }

  private createFormUnits() {
    this.fbs.set([
      new StringUnit({ key: `${prefix}_url`, label: 'URL' }),
      new StringUnit({ key: `${prefix}_userName`, label: 'User Name' }),
      new PasswordUnit({ key: `${prefix}_password`, label: 'Password' }),
      new StringUnit({ key: 'method', label: 'Method', display: () => this.type === 'autolog' }),
      // ...
    ]);
  }
}

// 3 个子类: SmallCellLogComponent / PacketCaptureComponent / AutoLogComponent
```

九、参数树组件 — 懒加载树 + WebSocket 实时刷新

9.1 懒加载树形结构

```
export class ParameterTreeComponent implements OnInit, OnDestroy {
  readonly rows = signal<AppTree[]>([]);

  // 初始加载根节点
  refresh() {
    this.service.getParameterTree(this.neId(), 'null').subscribe(data => {
      this.rows.set([ ...data, _expand: true ]]);
      this.getParameter(this.rows()[0]); // 加载第一层子节点
    });
  }

  // 懒加载子节点 — 展开时才请求
  getParameter(row: AppTree) {
    row._loading = true;
    this.service.getParameterTree(this.neId(), row.fullName)
      .pipe(finally(() => row._loading = false))
      .subscribe(data => {
```

```

        row.children = data.children?.sort((a, b) => a.name.localeCompare(b.name));
        row._expand = data.children !== null;
    });
}

// 展开/折叠事件
collapse({ data, event }) {
    data._expand = event;
    if (event && data.children!.length === 0) {
        this.getParameter(data); // 首次展开时加载
    }
}
}
}

type AppTree = Tree & { _expand?: boolean; _loading?: boolean };

```

9.2 WebSocket 实时刷新状态

```

// 订阅参数刷新状态
this.stomp = new StompService('parameterMonitor', 'parameterMonitor/' + identity);
this.stomp.watch().subscribe(msg => {
    if (msg.body === 'refreshing') {
        this.flag = { text: 'Refreshing', color: '#eb7836' }; // 橙色: 刷新中
    } else {
        this.flag = { text: `Refresh Time: ${new Date(parseInt(msg.body)).toISOString()}`,
            color: '#49b77a' }; // 绿色: 完成
    }
});

// 参数搜索 + 逐层展开定位
searchAndExpand(path: string) {
    this.collapseAll(); // 先折叠所有
    const hierarchyArray = path.split('.').map((_, i, arr) => arr.slice(0, i + 1).join('.'));
    this.expandHierarchy(hierarchyArray, 0); // 逐层展开到目标节点
}

private expandHierarchy(hierarchyArray: string[], index: number) {
    const node = this.findNodeByFullName(hierarchyArray[index], this.rows());
    if (node && !node._expand) {
        this.getParameter(node); // 懒加载
        // 等待加载完成后继续展开下一层
        const checkInterval = setInterval(() => {
            if (!node._loading && node.children?.length > 0) {
                clearInterval(checkInterval);
                this.expandHierarchy(hierarchyArray, index + 1);
            }
        }, 100);
    }
}

// 滚动到目标节点
private scrollToNode(fullName: string) {

```

```

setTimeout(() => {
  const element =
this.elementRef.nativeElement.querySelector(`span[title="${fullName}"]`);
  element?.scrollIntoView({ behavior: 'smooth', block: 'center' });
}, 300);
}

```

技术深度：

- **懒加载树**：只有展开时才请求子节点，避免一次性加载整棵树
- **递归搜索展开**：`expandHierarchy()` 递归加载每一层，`setInterval` 轮询等待加载完成
- **DOM 定位**：`scrollIntoView()` 平滑滚动到搜索结果
- **WebSocket 状态**：实时显示参数树刷新进度（refreshing → 完成时间）

十、节点监控 — 自动轮询 + 倒计时

```

export class NodeComponent extends Pagination<Status> {
  readonly refreshInterval = signal(60);

  override ngOnInit() {
    interval(1000) // 每秒触发
      .pipe(takeUntilDestroyed(this.destroyRef), startWith(-1))
      .subscribe(() => {
        if (this.refreshInterval() >= 60) { // 每 60 秒刷新数据
          this.refresh();
          this.refreshInterval.set(0); // 重置计数
        } else {
          this.refreshInterval.update(x => x + 1); // 计数递增
        }
      });
  }
}

// 模板中显示倒计时
// {{ 60 - refreshInterval() }}s 后刷新

```

设计要点：

- `interval(1000)` + `signal` 实现精确倒计时
- `startWith(-1)` 确保组件初始化时立即执行一次刷新
- `takeUntilDestroyed()` 组件销毁时自动取消订阅
- 用户可感知刷新节奏，避免"数据突然变化"的困惑

十一、批量操作 — 动态表单 + 任务管理

```

// Bulk 操作架构
BulkComponent (列表页)

```



```
└─ DynamicCommonTaskFormComponent (动态任务表单)
  └─ TaskBaseInfoComponent (任务基本信息: 名称/时间/状态)
    └─ TaskConditionInfoComponent (任务条件: 网元过滤/分组选择)
└─ ExecuteResultComponent (执行结果展示)
```

// 动态表单 - 根据任务类型动态渲染字段

```
@Component({
  imports: [TaskBaseInfoComponent, TaskConditionInfoComponent, ...]
})
export class DynamicCommonTaskFormComponent {
  data = inject<{ taskList: TaskList }>(NZ_MODAL_DATA);
  readonly taskList = signal<this.data.taskList>();
  readonly taskType = signal<this.data.taskList.taskType>();
  readonly conditionList = signal<this.data.taskList.filterConditions>();
  readonly versionFile = signal<this.data.taskList.versionFile>();
}
```

十二、再平衡管理 — 多策略网元迁移

Rebalance (再平衡模块)

```
└─ RebalanceTaskComponent (任务列表 - 6 种状态)
└─ RebalanceListComponent (再平衡列表 - 批量操作)
└─ LocalSwitchoverComponent (本地切换)
└─ RemoteSwitchoverComponent (远程切换)
└─ ClusterMigrateComponent (集群迁移)
```

// 共享子组件

```
component/
└─ RebalanceNeComponent (网元选择)
└─ MigrateComponent (迁移配置)
└─ ResultComponent (执行结果)
└─ ListComponent (通用列表)
```

任务状态机:

```
Not Running (0) → Executing (1) → Completed Successful (2)
                                   → Completed Failed (3)
                                   → Pause (4)
                                   → Partially Failed (5)
```

十三、告警高级功能

13.1 告警规则引擎

```
export class RuleComponent extends Pagination<AlarmRule> {
  readonly buttonType = signal<AlarmRuleType[]>([
    'Alarm Filtering',      // 告警过滤
    'Alarm Auto ACK',      // 告警自动确认
  ])
```

```

        'Alarm Forwarding',          // 告警转发
    ]);

    // 规则状态切换
    changeStatus(v: AlarmRule) {
        this.modal.confirm(
            DialogModal.builder()
                .setTitle(`${v.status ? 'Suspend' : 'Active'} ${v.type} Rule`)
                .setOk(() => this.service.changeRuleStatus(
                    v.id!, v.status ? 0 : 1,
                    v.status ? ROLE.alarm.rule.SUSPEND.M : ROLE.alarm.rule.ACTIVE.M
                ))
        );
    }

    // 规则编辑 - 联动网元分组树
    add(type: AlarmRuleType) {
        this.loadNeGroupTree().subscribe(neGroupTree => {
            this.modal.create({
                nzContent: FilterTemplateComponent,
                nzData: { rule: { type }, neGroupTree, mode: 'add' },
            });
        });
    }
}

```

13.2 告警导出自定义字段

```

// CustomExportComponent - 用户选择导出字段
@Component({
    template: `
        @for (field of exportFields; track field.key) {
            <nz-checkbox [formControlName]="field.key">{{ field.label }}</nz-checkbox>
        }
    `,
})
export class CustomExportComponent {
    // 支持 15+ 个导出字段: identity/raisedTime/perceivedSeverity/alarmID/eventType/...
    // 时区偏移自动计算
    body.timezoneOffset = new Date().getTimezoneOffset() / 6;
}

```

十四、系统设置 — 配置管理矩阵

```

Setting Config (系统配置)
├─ SystemConfigComponent (系统配置)
│   ├─ PasswordComponent (密码策略)
│   ├─ GlobalComponent (全局配置)
│   ├─ AemsNameComponent (AeMS 名称)
│   └─ GpsOffsetComponent (GPS 偏移)

```

```

|      └─ IopInventoryComponent (IOP 盘点)
|─ LogConfigComponent (日志配置)
|─ LdapConfigComponent (LDAP 配置)
|─ CwmpLogComponent (CWMP 日志配置)
|─ FemtoConfigComponent (Femto 配置)
|─ FileServerComponent (文件服务器)
|      └─ SmallCellLogComponent
|      └─ PacketCaptureComponent
|      └─ AutoLogComponent
└─ CapacityComponent (容量配置)
    └─ RebalanceConfigComponent (再平衡配置)
    └─ AlarmStorageComponent (告警存储配置)

```

配置模式统一：所有配置页继承 `FileServerBase` 或使用 `@axyom-ui/form` 的声明式表单，加载→编辑→保存流程完全一致。

十五、authInterceptor 的 Blob 错误处理

```

// 处理文件下载时的 Blob 错误（后端返回 JSON 错误但 Content-Type 是 application/octet-stream）
case 400:
case 500:
  const err = ev as HttpResponse;
  if (err.error instanceof Blob) {
    const reader = new FileReader();
    reader.readAsText(err.error, 'utf-8');
    reader.onload = () => {
      const t = JSON.parse(reader.result as string);
      modalService.error({ nzTitle: t.error, nzContent: t.message });
    };
  }
  // 后端自定义错误处理
  if (has(err.error, 'handleStatus') && err.error.handleStatus) {
    success = true; // 后端已处理，返回正常响应
  }

```

技术深度：

- **Blob→JSON 转换：** `FileReader.readAsText()` 将二进制 Blob 转为文本再解析
- **后端协同：** `handleStatus` 标志表示后端已处理错误，前端不需要再次抛出
- **错误通知：** `NZNotificationService` 区分 warning/error 类型

十六、扩展面试问答（5 题）

Q11: 右键菜单体系如何设计？

答：组合模式 + 观察者模式。BaseMenuService 提供路由导航和 refresh\$ Subject，四个子菜单服务（Maintenance/Operation/Radio/Log）各自封装业务逻辑并通过 getMenu() 返回菜单配置。ActiveListMenu 聚合所有子菜单。操作完成后通过 refresh\$.next() 通知列表刷新。

Q12: 参数树的懒加载和搜索定位如何实现？

答：懒加载：展开节点时才调用 getParameterTree(neId, fullName) 请求子节点，_expand/_loading 标志控制 UI 状态。搜索定位：searchAndExpand() 先折叠所有节点，然后 expandHierarchy() 递归逐层展开到目标路径，setInterval 轮询等待每层加载完成，最后 scrollToView() 平滑滚动。

Q13: CellBase 模板方法模式的好处？

答：8 个设备详情页（General/Upgrade/Diagnostics/Parameter 等）共享路由参数解析、设备信息加载、面包屑设置逻辑。子类只需重写 init() 方法实现具体业务。减少约 60% 的重复代码，统一维护入口。

Q14: 文件下载时 Blob 错误如何处理？

答：authInterceptor 检测 err.error instanceof Blob，用 FileReader.readAsText() 将 Blob 转为 JSON 解析错误信息。这是处理文件下载 API 返回 JSON 错误的特殊场景（Content-Type 与实际内容不匹配）。

Q15: 如何保证 30+ 个 API 服务的一致性？

答：① 统一继承 BaseApi，使用相同装饰器体系；② 每个 API 模块有独立的 type/ 目录定义接口类型；③ index.ts 统一导出；④ api/readme.md 规范目录结构和命名；⑤ 权限码通过 ROLE 常量统一管理，装饰器 URL 中直接引用 ROLE.xxx.M。

七、技术亮点速查表

#	亮点	关键词	代码位置
1	声明式 API 服务	装饰器、BaseApi、类型安全	api/**/*.service.ts
2	LRU 路由缓存	RouteReuseStrategy、滚动恢复	customReuseStrategy.ts
3	位编码权限	RBAC、ACL、三层控制	core/model/role.ts
4	十万级设备地图	BBOX 裁剪、聚合、WMS	routes/dashboard/osm-map/
5	精确 Loading	HttpInterceptor、Signal	core/interceptor/loading.interceptor.ts
6	SessionStorage 装饰器	Object.defineProperty	core/decorator/session-storage.ts
7	STOMP WebSocket	实时推送、强制登出	core/stomp.service.ts
8	异步导出轮询	expand/takeWhile、流式下载	core/utils/download-file.ts
9	GeoHA 双活	ID 哈希分片、无状态判断	core/service/status.service.ts
10	分页基类泛型	Pagination<T>、computed 过滤	core/model/page.ts

#	亮点	关键词	代码位置
11	声明式表单	FormBase、DynamicModal	share/component/
12	全局搜索防抖	BehaviorSubject、debounceTime	page/layout/header/search/
13	右键菜单组合	Composite 模式、refresh\$ 通信	manage/active-list/*-menu.ts
14	CellBase 模板方法	路由参数解析、设备信息加载	manage/active-list/component/cell.base.ts
15	参数树懒加载	懒加载树、WebSocket 刷新、搜索定位	manage/active-list/parameter/tree/
16	节点监控倒计时	interval + signal 精确计时	routes/monitor/node/
17	批量任务动态表单	动态渲染条件字段	manage/bulk/
18	再平衡状态机	6 种任务状态、多策略迁移	manage/rebalance/
19	Blob 错误处理	FileReader + Blob→JSON	core/interceptor/auth.interceptor.ts
20	FileServerBase 配置	基类 + 子类动态渲染	routes/setting/config/file-server/